CORE JAVA · OOP · COLLECTIONS · CODING ROUND

THE JAVA INTERVIEW PLAYBOOK

From Core Java Fundamentals to Coding Interview Confidence



100	30	30	3
Interview Q&A	Output Questions	Coding Problems	Mock Interviews

2026 EDITION

Independent Java Learning Resource

Beginner to intermediate · Examples verified on OpenJDK 21 · Not affiliated with or endorsed by Oracle

Contents

Click any line to jump to that chapter.

PART I JAVA FOUNDATIONS 5

01 Welcome to Java 5
02 Variables, Data Types, and Operators 9
03 Conditional Statements and Loops 15
04 Methods and Recursion 20

PART II OBJECT-ORIENTED PROGRAMMING 24

05 Classes and Objects 24
06 The Four Pillars of OOP 28
07 Advanced OOP Essentials 33

PART III STRINGS, ARRAYS AND COLLECTIONS 37

08 Arrays 37
09 Strings 41
10 Java Collections Framework 47

PART IV EXCEPTIONS AND IMPORTANT FEATURES 52

11 Exception Handling 52
12 Essential Java Concepts 57

PART V INTERVIEW PREPARATION 63

13 100 Java Interview Questions and Answers 64
14 30 Java Output-Based Questions 72
15 30 Essential Coding Problems 80

PART VI REVISION AND PRACTICE 104

16 One-Day Java Interview Revision Plan 104
17 Mock Interview Workbook 108
18 Final Cheat Sheets 115

How to Use This Book

This is a **workbook**, not a novel. The fastest way to learn Java for an interview is to read a concept, type the example yourself, change it, break it, and then explain it aloud. Work through the parts in order, or jump to Part V if you are short on time.



The teaching pattern

Most topics follow the same rhythm so you always know where you are:

1 WHAT & WHY
A plain-English definition, then where the concept matters in real programs and interviews.

2 SEE IT
An analogy, a diagram or table, then readable Java code with a line-by-line breakdown.

3 TRACE IT
A dry-run table that shows how variables change step by step.

4 AVOID & PRACTISE
Common mistakes, interview tips, and exercises with answers.

Colour-coded labels

Label	Meaning
CONCEPT	Core definition or rule to remember
EXAMPLE / DRY RUN	Worked code and a step-by-step trace of values
INTERVIEW TIP / TRAP	How the topic is asked, and the wrong answers candidates give
COMMON MISTAKE	Errors beginners make, with the fix
PRACTICE / SOLUTION	Exercises first; answers in the green box that follows

Choose your study plan

Time available	Suggested route
1 day	Chapter 16 (hour-by-hour plan), then Chapter 18 cheat sheets, then one mock interview from Chapter 17.
1 week	Day 1: Ch 1-4 | Day 2: Ch 5-7 | Day 3: Ch 8-10 | Day 4: Ch 11-12 + Ch 14 | Day 5: Ch 13 | Day 6: Ch 15 | Day 7: Ch 17 + Ch 18.
3 weeks	One chapter per day in Parts I-IV (type every example), then 10 Q&A, 5 output questions and 3 coding problems per day from Part V, then the three mocks.

INTERVIEW TIP

Interviewers care about **how you think**. When you solve a problem, say your approach aloud, state the time complexity, and mention edge cases before you write code. Every coding problem in Chapter 15 is laid out that way.

CHAPTER

06 The Four Pillars of OOP

Encapsulation, abstraction, inheritance and polymorphism, with the answers interviewers expect.

STUDY TIME 50 min HOW OFTEN ASKED

IN THIS CHAPTER YOU WILL

- Define each of the four pillars
- Reuse code with inheritance
- Compare overloading with overriding
- Protect data with encapsulation
- Override methods and see runtime dispatch
- Answer the classic OOP questions

The four pillars at a glance

ENCAPSULATION

Hide data, expose methods

```
private fields + getters
```

ABSTRACTION

Hide complexity, show essentials

```
abstract class / interface
```

INHERITANCE

Reuse and extend a parent

```
class Dog extends Animal
```

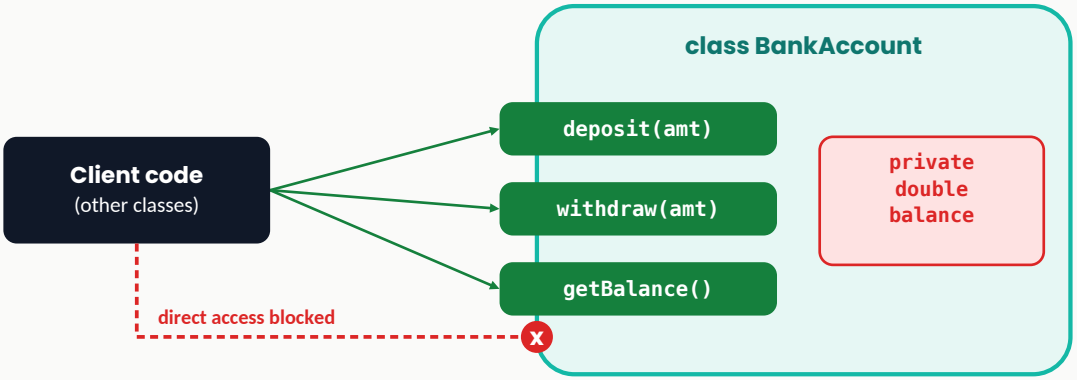
POLYMORPHISM

One call, many behaviours

```
Animal a = new Dog()
```

Pillar	In one sentence	Java tool	Real-world analogy
Encapsulation	Bundle data with the methods that use it and hide the data	`private` fields + public getters / setters	A medicine capsule hides its contents
Abstraction	Show what an object does, hide how it does it	`abstract` classes, interfaces	Driving a car without knowing the engine
Inheritance	A class acquires fields and methods of a parent	`extends`	A child inherits traits from parents
Polymorphism	One call, different behaviour depending on the object	Overloading, overriding	A "Play" button works for music or video

Encapsulation



Only public methods can touch the data. They validate every change (e.g. reject negative deposits).

Encapsulation.java (part 1 of 2) JAVA

```java
class BankAccount {
    private double balance;                          // hidden data
    void deposit(double amt) { if (amt > 0) balance += amt; }
    boolean withdraw(double amt) {
        if (amt > 0 && amt <= balance) { balance -= amt; return true; }
        return false;                                // validation protects the data
    }
    double getBalance() { return balance; }
}
public class Main {
```

Encapsulation.java (part 2 of 2) JAVA

```java
    // ... continued from part 1
    public static void main(String[] args) {
        BankAccount acct = new BankAccount();
        acct.deposit(500);
        System.out.println(acct.withdraw(200));
        System.out.println(acct.withdraw(900));
        System.out.println(acct.getBalance());
    }
}
```

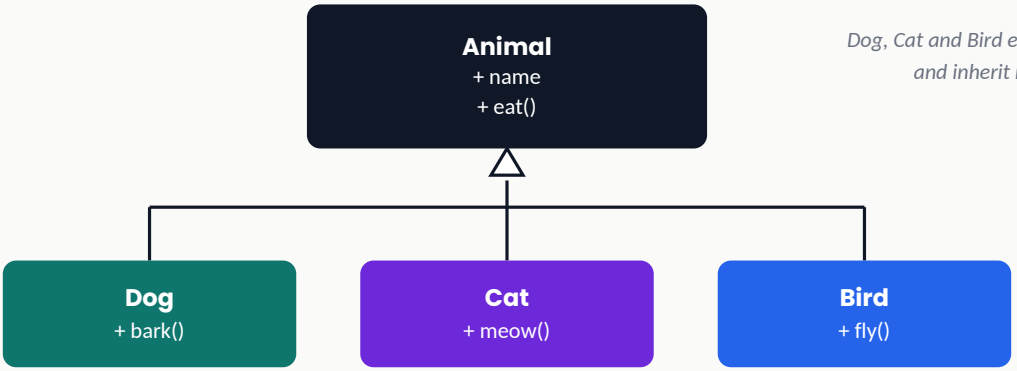
CONSOLE OUTPUT

```
true
false
300.0
```

WHY IT MATTERS

Because the field is `private`, no other class can set `balance = -1000`. Every change must pass through a method that enforces the rules, so you can change the internals later without breaking callers.

Inheritance



Dog, Cat and Bird each IS-A Animal and inherit name and eat().

Inheritance models an **IS-A** relationship: a `Dog` **is an** `Animal`. The child (subclass) gets the parent's (superclass) non-private members and may add or **override** behaviour. Java allows **single inheritance of classes** (one parent), but a class can implement many interfaces.

Polymorphism and method overriding

Polymorphism.java (part 1 of 2) — JAVA

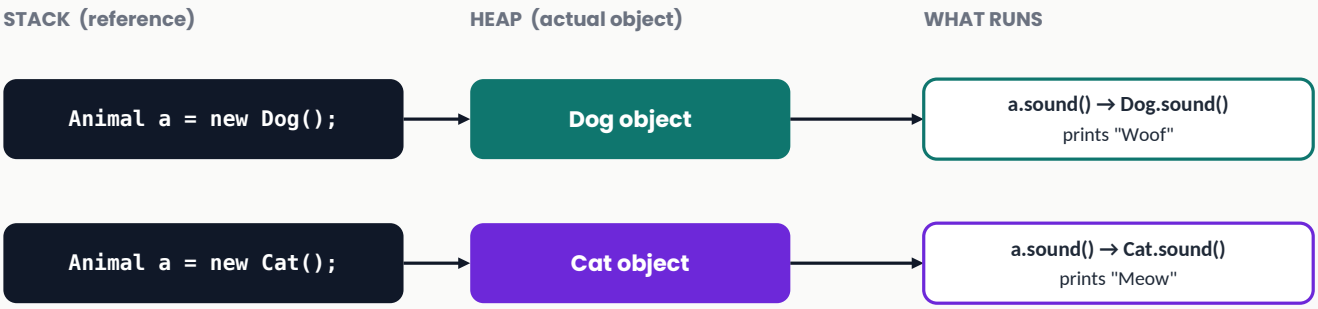
```java
class Animal {
    String name = "Animal";
    void sound() { System.out.println("Some sound"); }
    void eat()   { System.out.println("eating"); }
}
class Dog extends Animal {
    String name = "Dog";
    @Override void sound() { System.out.println("Woof"); }
    void fetch() { System.out.println("fetching"); }
}
class Cat extends Animal {
    @Override void sound() { System.out.println("Meow"); }
}
public class Main {
```

Polymorphism.java (part 2 of 2) — JAVA

```java
    // ... continued from part 1
    public static void main(String[] args) {
        Animal a = new Dog();          // parent reference, child object
        a.sound();                     // decided at RUN time -> Dog's version
        a.eat();                       // inherited
        System.out.println(a.name);    // fields are NOT polymorphic -> Animal's field
        a = new Cat();
        a.sound();
        Animal[] zoo = { new Dog(), new Cat(), new Animal() };
        for (Animal x : zoo) x.sound();
    }
}
```

CONSOLE OUTPUT

```
Woof
eating
Animal
Meow
Woof
Meow
Some sound
```



The reference type (Animal) is checked at compile time; the object type decides WHICH override runs at run time.

CONCEPT

Dynamic method dispatch: when an overridden method is called through a parent reference, the JVM looks at the **actual object** at run time and runs that class's version. The compiler only checks that the **reference type** has the method: `a.fetch()` would not compile because `Animal` has no `fetch()`.

Overloading vs overriding

	Overloading	Overriding
Where	Same class (or subclass)	Subclass redefines a superclass method
Signature	Same name, **different** parameters	Same name, **same** parameters
Return type	Can differ	Same, or a subtype (covariant)
Resolved	**Compile time** (static / early binding)	**Run time** (dynamic / late binding)
Access / exceptions	No restriction	Cannot reduce visibility or throw broader checked exceptions
`static`, `final`, `private`	Can be overloaded	**Cannot** be overridden (static is hidden, not overridden)

COMMON MISTAKE

Forgetting `@Override`: a typo such as `sond()` silently creates a new method instead of overriding. `@Override` makes the compiler check.

Assuming fields are polymorphic: only instance **methods** are dispatched at run time. Fields follow the reference type.

Frequently asked interview questions

Question	Short answer
Why is multiple inheritance of classes not allowed?	It creates the **diamond problem** (ambiguity about which parent's method to inherit). Interfaces avoid it by having no state.
Can we override a `private` or `static` method?	No. A private method is not visible to the subclass; a static method is **hidden**, not overridden.
Difference between IS-A and HAS-A?	IS-A is inheritance (`Dog extends Animal`). HAS-A is composition (`Car` has an `Engine` field).
Is method overloading polymorphism?	Yes, **compile-time** polymorphism. Overriding is **run-time** polymorphism.
What is the root class of every Java class?	`java.lang.Object`. It provides `toString()`, `equals()`, `hashCode()` and more.

INTERVIEW TIP

Answer "What is polymorphism?" with the phrase **"one interface, many implementations"**, then immediately give the `Animal a = new Dog()` example. A concrete example beats a definition.

Practice

PRACTICE

1. Which pillar does making fields `private` demonstrate?

2. `Animal a = new Dog();` Which method runs for `a.sound()` and when is that decided?

3. Will `class A { void f() {} } class B extends A { int f() { return 1; } }` compile?

SOLUTION

1. Encapsulation.

2. `Dog.sound()`, decided at **run time** from the object's real class.

3. No. The signature `f()` matches, but the return type `int` is incompatible with `void`.

CHAPTER RECAP

Encapsulation: private data plus validating methods. Abstraction: hide complexity. Inheritance: IS-A reuse. Polymorphism: behaviour chosen at run time.

Overridden **methods** follow the object at run time; **fields** and static methods follow the reference type.

Use `@Override`; keep the signature and never reduce visibility.

CHAPTER

13

100 Java Interview Questions and Answers

Ten categories, one hundred answers: cover the answer, speak yours aloud, then compare.

STUDY TIME 3 hours **HOW OFTEN ASKED** ●●●●●

IN THIS CHAPTER YOU WILL

- Answer fundamentals confidently
- Compare similar concepts clearly
- Explain JVM and memory basics
- Predict outputs and spot traps
- Handle scenario-based questions
- Mention modern Java features accurately

JAVA FUNDAMENTALS

1 What is Java, and why is it called platform-independent?

Java is a general-purpose, object-oriented language. The compiler turns source code into **bytecode**, and every operating system has a JVM that runs that same bytecode. So the program is portable even though each JVM is platform-specific.

2 What is the difference between JDK, JRE and JVM?

JVM runs bytecode. **JRE** = JVM + core libraries. **JDK** = JRE + development tools (`javac` , `jar` , `javadoc` , `jdb`). Developers need the JDK; modern installs are normally a full JDK.

3 What is bytecode and who executes it?

Bytecode is the platform-neutral instruction set stored in `.class` files. The **JVM** executes it: first interpreting it, then compiling hot code to machine code with the JIT compiler.

4 What is the JIT compiler?

The Just-In-Time compiler is part of the JVM. It watches which methods run often ("hot spots") and compiles them to native machine code at run time, so long-running Java programs become much faster than pure interpretation.

5 Why is main declared public static void main(String[] args)?

`public` : the JVM must reach it from outside. `static` : it runs before any object exists. `void` : nothing is returned to the JVM. `String[] args` : command-line arguments. Changing the signature means the JVM will not treat it as the entry point.

6 Is Java a purely object-oriented language?

No. It has **eight primitive types** (`int` , `char` , ...) that are not objects, and `static` members that belong to classes. Wrapper classes (`Integer`) bridge the gap.

7 Can one .java file contain more than one class?

Yes. But at most one can be `public` , and that public class name must match the file name. Each class gets its own `.class` file after compilation.

8 What is the classpath?

The list of directories and JAR files where the JVM and compiler search for classes. Set it with `-cp` / `-classpath` or the `CLASSPATH` variable. A missing entry causes `ClassNotFoundException` .

9 Is Java compiled or interpreted?

Both. `javac` compiles source to bytecode (compiled step); the JVM interprets and JIT-compiles bytecode at run time (interpreted step).

10 What do package and import do? Does import load classes?

`package` names the group a class belongs to. `import` only lets you use a **short name** (`List` instead of `java.util.List`); it does not load anything. `java.lang` is imported automatically.

DATA TYPES AND OPERATORS

11 **Name the eight primitive types and their sizes.**

`byte` 8, `short` 16, `int` 32, `long` 64, `float` 32, `double` 64 (bits), `char` 16 (unsigned), and `boolean` (true/false, size not fixed by the specification).

12 **Why does b = b + 1 fail for a byte, but b += 1 works?**

```
byte b = 10;
b = b + 1;   // error
b += 1;      // OK
```

`b + 1` is promoted to `int`, and assigning an `int` to a `byte` needs a cast. The compound `+=` includes an implicit cast back to `byte`.

13 **What is the difference between float and double?**

`float` is 32-bit (about 7 digits); `double` is 64-bit (about 15-16 digits). A decimal literal such as `3.14` is a `double`; write `3.14f` for a `float`.

14 **What is the output?**

```
System.out.println(
  10 + 20 + "Java" + 10 + 20);
```

Evaluated left to right: `10 + 20 = 30`, then string concatenation takes over. Output: **30Java1020**.

15 **What is the difference between i++ and ++i?**

`i++` yields the **old** value, then increments. `++i` increments first, then yields the **new** value. As a standalone statement they behave the same.

16 **What happens when an int overflows?**

It silently **wraps around**: `Integer.MAX_VALUE + 1` becomes `-2147483648`. Use `long`, or `Math.addExact()` which throws `ArithmeticException`.

17 **What is the difference between & and &&?**

`&&` is logical and **short-circuits**: if the left side is false the right side is skipped. `&` always evaluates both sides, and also works bitwise on integers.

18 **What is the difference between >> and >>>?**

`>>` is the **signed** right shift (keeps the sign bit). `>>>` is the **unsigned** shift (fills with zeros). For `-8`, `>> 1` gives `-4`; `>>> 1` gives a large positive number.

19 **Why is 0.1 + 0.2 == 0.3 false? What should money use?**

Binary floating point cannot represent 0.1 exactly, so the sum is `0.30000000000000004`. For money use `BigDecimal` (or integer cents); compare doubles with a small tolerance.

20 **What are 5 / 2, 5 % 2, -5 % 2 and 5 / 2.0?**

`2`, `1`, `-1` and `2.5`. Integer division truncates; the remainder takes the sign of the left operand; one `double` operand forces decimal division.